

TALLER: Explorando las diferencias entre MySQL y PostgreSQL con Docker

Objetivo general

Reconocer las principales diferencias entre los motores de base de datos MySQL y PostgreSQL, ejecutarlos en contenedores Docker, y explorar sus comandos básicos en la terminal interactiva de PostgreSQL, complementando con el uso de un cliente gráfico (pgAdmin o DBeaver).

Objetivos específicos

- Comprender los conceptos generales de MySQL y PostgreSQL.
- Comparar su estructura, características, ventajas y comandos principales.
- Explorar los comandos básicos desde la terminal interactiva (psql).
- Conectarse a la base de datos con un cliente gráfico externo (pgAdmin o DBeaver).

Parte 1. Consulta guiada

Investiga los siguientes aspectos sobre MySQL y PostgreSQL, consúltalos en fuentes confiables (documentación oficial, blogs técnicos, etc.) y completa la siguiente tabla. Adjunta una imagen o captura de la fuente consultada o del entorno donde verificaste la información.

Aspecto a comparar	MySQL	PostgreSQL
Tipo de sistema (relacional / mixto)		
Licencia		
Enfoque principal		
Tipos de datos admitidos		
Integridad referencial		
Soporte de JSON y datos complejos		
Soporte para funciones y procedimientos		
Nivel de cumplimiento del estándar SQL		
Extensiones disponibles		
Uso recomendado		

Parte 2. Exploración práctica con Docker y PostgreSQL

Realiza la configuración de tu entorno Docker para levantar una instancia de PostgreSQL. Registra los pasos ejecutados y adjunta un pantallazo de la ejecución exitosa.

Espacio para evidencias:

Descargamos la imagen de PostgresSql

TALLER BASES DE DATOS

ING. WILLIAM ALEXANDER MATALLANA PORRAS

```
PS C:\Users\SamuelFernandoForero> docker pull postgres:latest
latest: Pulling from library/postgres
8a9c24e23f88: Pull complete
28b206cbbc14: Pull complete
168b3ade331e: Pull complete
6458b7f41c65: Pull complete
0a51ed68fa52: Pull complete
ce1261c6d567: Pull complete
8d0a13cb166d: Pull complete
1d5017cf452d: Pull complete
e1b18b5359f0: Pull complete
5773151508cd: Pull complete
180db792316f: Pull complete
ef07360e404d: Pull complete
30b4b10fcf1d: Pull complete
8551209c5a1e: Pull complete
Digest: sha256:1d288494853e244e7a78d87b3526e650e5221c622f9768ecac9313d0874a9c39
Status: Downloaded newer image for postgres:latest
docker.io/library/postgres:latest
PS C:\Users\SamuelFernandoForero> |
```

Creamos el contenedor:

```
PS C:\Users\SamuelFernandoForero> docker run -d --name postgres-dev -p 5433:5432 -e P
OSTGRES_PASSWORD=1234 -e POSTGRES_USER=admin -v D:/Personal/LineaProfundizacionDatosT
res/MyPostgresVolumen postgres:latest
8a5d3ad774474edf893497f73f0f0bd02a17c60ac966e895b03523b59cd5531e
PS C:\Users\SamuelFernandoForero> |
```

Containers / postgres-dev

postgres-dev

<  8a5d3ad77447  postgres:latest

5433:5432

STATUS
Running (1 minute ago)

Logs Inspect Bind mounts Exec Files Stats

waiting for server to shut down....2025-09-29 23:56:34.425 UTC [49] LOG: received fast shutdown request

2025-09-29 23:56:34.428 UTC [49] LOG: aborting any active transactions

2025-09-29 23:56:34.430 UTC [49] LOG: background worker "logical replication launcher" (PID 58) exited with exit code 1

2025-09-29 23:56:34.430 UTC [53] LOG: shutting down

2025-09-29 23:56:34.433 UTC [53] LOG: checkpoint starting: shutdown immediate

2025-09-29 23:56:34.679 UTC [53] LOG: checkpoint complete: wrote 943 buffers (5.8%), wrote 3 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0.020 s, sync=0.216 s, total=0.250 s; sync files=303, longest=0.004 s, average=0.001 s; distance=4352 kB, estimate=4352 kB; lsn=0/1B9FBA0, redo lsn=0/1B9FBA0

2025-09-29 23:56:34.703 UTC [49] LOG: database system is shut down

done

server stopped

PostgreSQL init process complete; ready for start up.

2025-09-29 23:56:34.768 UTC [1] LOG: starting PostgreSQL 18.0 (Debian 18.0-1.pgdg13+3) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit

2025-09-29 23:56:34.769 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432

2025-09-29 23:56:34.769 UTC [1] LOG: listening on IPv6 address ":::", port 5432

2025-09-29 23:56:34.780 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"

2025-09-29 23:56:34.792 UTC [71] LOG: database system was shut down at 2025-09-29 23:56:34 UTC

2025-09-29 23:56:34.798 UTC [1] LOG: database system is ready to accept connections

Parte 3. Exploración en la terminal interactiva (psql)

Accede al contenedor de PostgreSQL y explora la terminal interactiva. Ejecuta los comandos básicos para gestionar bases de datos, usuarios y tablas. Describe con tus palabras qué hace cada acción y adjunta la evidencia visual (pantallazo de la terminal).

```
PS C:\Users\SamuelFernandoForero> docker exec -it postgres-dev bash
root@8a5d3ad77447:/# psql -U admin
psql (18.0 (Debian 18.0-1.pgdg13+3))
Type "help" for help.

admin=# |
```

Acción / Comando explorado	MSQL	PostgreSQL	Descripción o evidencia (pantallazo PostgreSQL)
Listar bases de datos	show databases;	\l	
Crear una base de datos	create database <nombre>;	create database <nombre>;	
Conectarse a una base específica	use database <nombre>	\c <nombre>	
Listar usuarios	SELECT User, Host FROM mysql.user;	\du	
Crear un usuario nuevo	CREATE USER 'nuevo_usuario'@'%' IDENTIFIED BY 'mi_password'; GRANT ALL PRIVILEGES ON *.* TO 'nuevo_usuario'@'%' WITH GRANT OPTION; FLUSH PRIVILEGES;	CREATE USER nuevo_usuario WITH PASSWORD 'mi_password';	
Crear una tabla	CREATE TABLE persona (id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT NULL, edad INT, email VARCHAR(150) UNIQUE, fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP);	CREATE TABLE persona (id SERIAL PRIMARY KEY, nombre VARCHAR(100) NOT NULL, edad INT, email VARCHAR(150) UNIQUE, fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP);	
Insertar datos en una tabla	INSERT INTO persona (nombre, edad, email) VALUES ('Juan Pérez', 30, 'juan@example.com');	INSERT INTO persona (nombre, edad, email) VALUES ('Juan Pérez', 30, 'juan@example.com');	

TALLER BASES DE DATOS
ING. WILLIAM ALEXANDER MATALLANA PORRAS

Consultar registros	select * from <tabla>;	select * from tabla	
Actualizar registros	UPDATE persona SET edad = 31, email = 'juan.perez@example.co m' WHERE id = 1;	UPDATE persona SET edad = 31, email = 'juan.perez@exam ple.com' WHERE id = 1;	
Eliminar registros	DELETE FROM persona WHERE id = 1;	DELETE FROM persona WHERE id = 1;	

Pantallazo 1:

```
admin=# \l
```

List of databases									
Name	Owner	Encoding	Locale	Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
admin	admin	UTF8	libc		en_US.utf8	en_US.utf8			
postgres	admin	UTF8	libc		en_US.utf8	en_US.utf8			
template0	admin	UTF8	libc		en_US.utf8	en_US.utf8			=c/admin +
template1	admin	UTF8	libc		en_US.utf8	en_US.utf8			admin=CTc/admin +

```
(4 rows)
admin=#
```

Pantallazo 2:

```
admin=# create database prueba;
CREATE DATABASE
admin=#
```

Pantallazo 3:

```
admin=# \c prueba
You are now connected to database "prueba" as user "admin".
prueba=#
```

Pantallazo 4:

```
prueba=# \du
```

List of roles	
Role name	Attributes
admin	Superuser, Create role, Create DB, Replication, Bypass RLS

```
prueba=#
```

Pantallazo 5:

```
prueba=# CREATE USER nuevo_usuario WITH PASSWORD 'password';
CREATE ROLE
prueba=#
```

Pantallazo 6:

```
prueba=# CREATE TABLE persona (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    edad INT,  
    email VARCHAR(150) UNIQUE,  
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
CREATE TABLE  
prueba=# |
```

Pantallazo 7:

```
prueba=# INSERT INTO persona (nombre, edad, email)  
VALUES ('Juan Pérez', 30, 'juan@example.com');  
INSERT 0 1  
prueba=# |
```

Pantallazo 8:

```
prueba=# select * from persona;  
 id | nombre | edad | email | fecha_creacion  
-----  
  1 | Juan Pérez |  30 | juan@example.com | 2025-09-30 00:34:39.846464  
(1 row)  
  
prueba=# |
```

Pantallazo 9:

```
prueba=# UPDATE persona  
SET edad = 31, email = 'juan.perez@example.com'  
WHERE id = 1;  
UPDATE 1  
prueba=# |
```

Pantallazo 10:

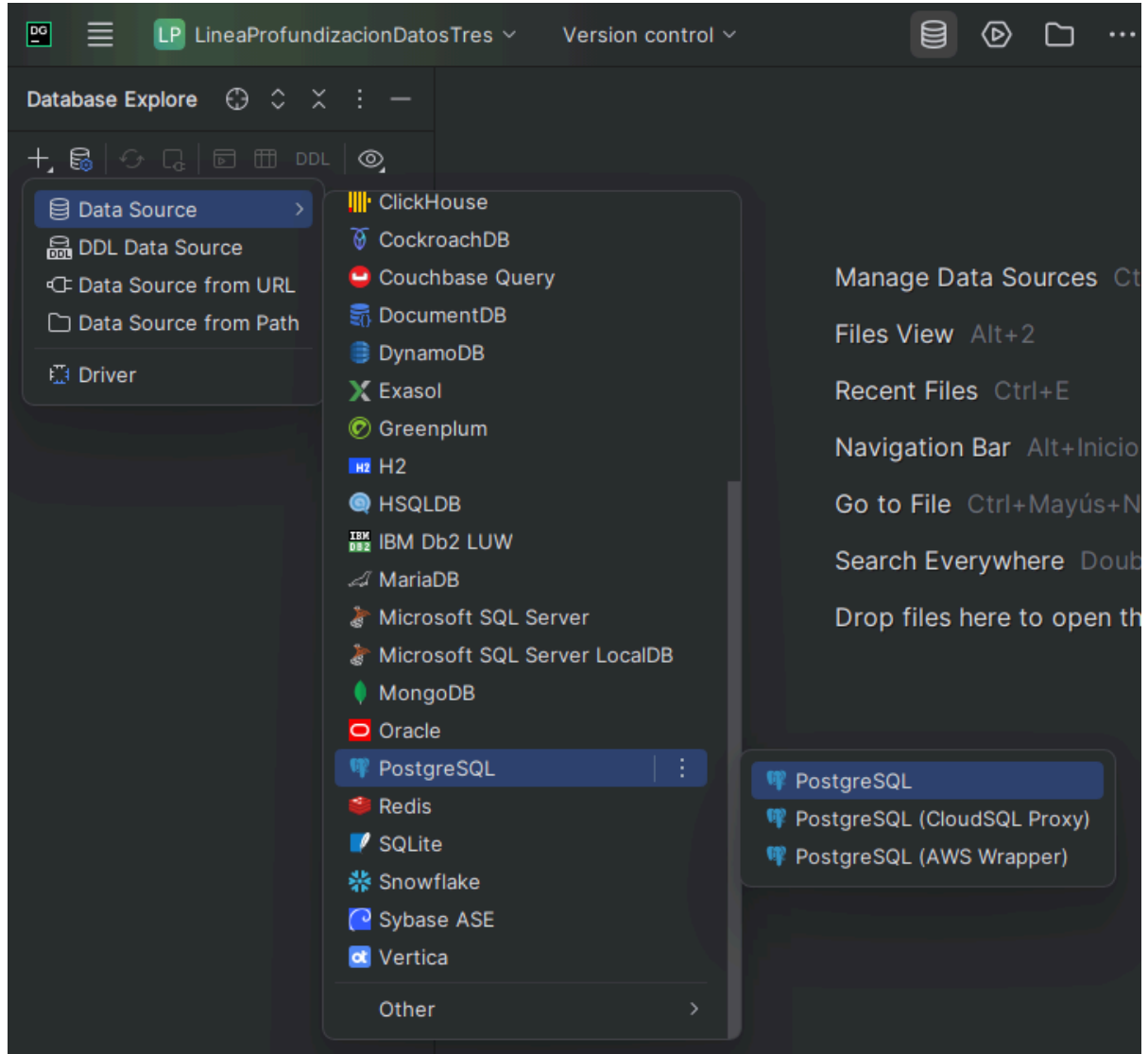
```
prueba=# DELETE FROM persona  
WHERE id = 1;  
DELETE 1  
prueba=# |
```

Parte 4. Conexión con cliente gráfico (pgAdmin o DBeaver)

Conéctate a la base de datos PostgreSQL utilizando pgAdmin o DBeaver. Realiza la conexión, crea una base de datos de prueba y explora sus tablas y registros. Anexa pantallazos de la conexión y describe brevemente el proceso.

Evidencia visual:

Se conecta con DataGrip de JetBrains



TALLER BASES DE DATOS
ING. WILLIAM ALEXANDER MATALLANA PORRAS

Data Sources and Drivers

Data Sources | Drivers | D v

Project Data Sources

- LineaTres
- LineaTresPostgres**

Problems

Name: LineaTresPostgres [Create DDL Mapping](#)

Comment:

General | Options | SSH/SSL | Schemas | Advanced

Connection type: default Driver: PostgreSQL [More Options](#)

Host: localhost Port: 5433

Authentication: User & Password

User: admin

Password: Save: Forever

Database: postgres

URL: jdbc:postgresql://localhost:5433/postgres
Overrides settings above

[Test Connection](#) PostgreSQL

[OK](#) [Cancel](#) [Apply](#)

LineaTresPostgres

- postgres (1 of 1)
 - public
 - Database O...
 - Server Object...

> ☐ All databases [add pattern](#)

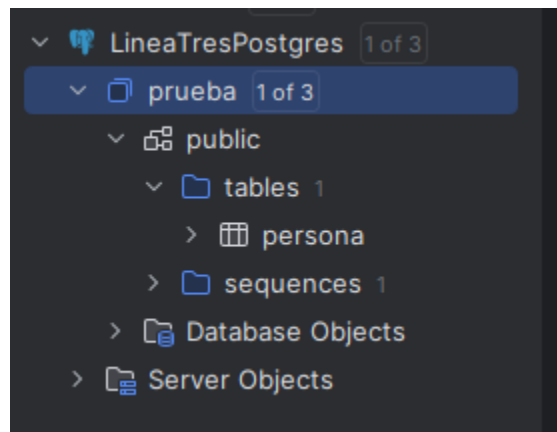
> ☐ Default database (postgres)

> ☐ admin

> ☒ postgres (Default database)

> ☐ prueba

- ☐ All schemas [add pattern](#)
- ☐ Default schema



Parte 5. Actividades de aplicación

Realiza las siguientes tareas en tu entorno PostgreSQL y documenta los resultados con pantallazos o descripciones breves.

Modelo Relacional (versión reducida)

1. Autor

Campo	Tipo de Dato	Clave	Descripción
id_autor	INT	PK	Identificador del autor
nombre	VARCHAR(120)		Nombre completo del autor

nacionalidad	VARCHAR(80)		Nacionalidad (opcional)
--------------	-------------	--	-------------------------

```
PostgresPractica.sql x
[Icons] Tx: Auto [Playground]
1 CREATE TABLE Autor (
2     id_autor serial primary key,
3     nombre varchar(120),
4     nacionalidad varchar(80)
5 );
6
7
```

2. Libro

Campo	Tipo de Dato	Clave	Descripción
id_libro	INT	PK	Identificador del libro
titulo	VARCHAR(200)		Título del libro
isbn	VARCHAR(20)		Código ISBN (único)
anio_publicacion	INT		Año de publicación
id_autor	INT	FK	Autor del libro (1 autor por libro)
stock	INT		Unidades disponibles para préstamo (>=0)

tables 3

- autor
- libro
 - columns 6
 - keys 1
 - foreign keys 1
 - fk_autor (id_autor)
 - indexes 1
- persona
- sequences 1

Database ObjectsServer Objects

```
6
7 CREATE TABLE Libro (
8     id_libro serial primary key,
9     titulo varchar(200) not null ,
10    isbn varchar(20) not null ,
11    anio_publicacion int not null,
12    id_autor int,
13    stock int,
14    CONSTRAINT fk_autor FOREIGN KEY (id_autor)
15        REFERENCES Autor(id_autor)
16        ON UPDATE CASCADE
17        ON DELETE SET NULL
18 );
```

3. Usuario

Campo	Tipo de Dato	Clave	Descripción
id_usuario	INT	PK	Identificador del usuario
documento	VARCHAR(30)		Documento de identidad (único)
nombre	VARCHAR(120)		Nombre completo
email	VARCHAR(120)		Correo electrónico (opcional)

```
19
20 ✓ CREATE TABLE Usuario(
21     id_usuario serial primary key ,
22     documento varchar(30) not null unique ,
23     nombre varchar(120) not null ,
24     email varchar(120)
25 );
```

4. Prestamo

(Un registro por libro prestado a un usuario. Si un usuario lleva 2 libros, se crean 2 filas.)

Campo	Tipo de Dato	Clave	Descripción
id_prestamo	INT	PK	Identificador del préstamo
id_usuario	INT	FK	Usuario que realiza el préstamo
id_libro	INT	FK	Libro prestado
fecha_prestamo	DATE		Fecha del préstamo

fecha_devolucion_max	DATE		Fecha límite de devolución
fecha_devuelto	DATE		Fecha de devolución efectiva (nullable)
estado	VARCHAR(15)		ABIERTO / CERRADO

```
CREATE TABLE Prestamo (  
    id_prestamo serial primary key ,  
    id_usuario int not null ,  
    id_libro int not null ,  
    fecha_prestamo DATE not null ,  
    fecha_devolucion_max DATE not null ,  
    fecha_devuelto DATE,  
    estado varchar(15)  
);  
  
ALTER TABLE Prestamo ADD CONSTRAINT fk_usuario FOREIGN KEY (id_usuario)  
    REFERENCES Usuario(id_usuario) ON UPDATE CASCADE ON DELETE SET NULL;  
  
ALTER TABLE Prestamo ADD CONSTRAINT fk_libro FOREIGN KEY (id_libro)  
    REFERENCES Libro(id_libro) ON UPDATE CASCADE ON DELETE SET NULL;
```

Relaciones

Autor — Libro

- Relación: Un autor tiene muchos libros.
- Cardinalidad: 1 a N.

Libro — Prestamo

- Relación: Un libro puede estar presente en muchos préstamos (una fila por unidad prestada).
- Cardinalidad: 1 a N.
- Regla de negocio sugerida: solo permitir préstamo si stock > 0 y al prestar disminuir stock en 1; al devolver, incrementar stock en 1.

Usuario — Prestamo

- Relación: Un usuario puede tener muchos préstamos.
- Cardinalidad: 1 a N.

Consultas a realizar

1. Libros con su autor

Columnas: titulo, nombre_autor, anio_publicacion.

```
44 ✓ SELECT l.titulo, a.nombre, l.anio_publicacion from Libro as l
45 INNER JOIN Autor A 1..n<->1: on A.id_autor = l.id_autor;
```

Output Result 5 x

	titulo	nombre	anio_publicacion
1	Libro prueba uno	Pepito	2001
2	Libro prueba dos	Pepita	2005

2. Préstamos abiertos con datos del usuario y del libro

Columnas: id_prestamo, usuario, titulo_libro, fecha_prestamo, fecha_devolucion_max, estado.

```
✓ SELECT p.id_prestamo,
      u.nombre,
      u.documento,
      l.titulo,
      p.fecha_prestamo,
      p.fecha_devolucion_max,
      p.estado
FROM prestamo as p
inner join Libro l on p.id_libro = l.id_libro
inner join Usuario u on p.id_usuario = u.id_usuario
WHERE p.estado = 'ABIERTO';
```

Output Result 6 x

id_prestamo	nombre	documento	titulo	fecha_prestamo	fecha_devolucion_max	estado
3	Nombre 2	4324324	Libro prueba uno	2025-09-22	2025-09-30	ABIERTO
1	Nombre 1	12010201	Libro prueba uno	2025-09-08	2025-10-15	ABIERTO

3. Historial de préstamos de un usuario (por documento)

Dado un documento, listar titulo_libro, fecha_prestamo, fecha_devuelto, estado.

TALLER BASES DE DATOS
ING. WILLIAM ALEXANDER MATALLANA PORRAS

```
SELECT p.id_prestamo, l.titulo, p.fecha_prestamo,
       p.fecha_devuelto, p.estado FROM prestamo AS p
inner join Libro L on p.id_libro = L.id_libro
inner join Usuario u on p.id_usuario = u.id_usuario
WHERE u.documento = '12010201';
```

Output Result 7

id_prestamo	titulo	fecha_prestamo	fecha_devuelto	estado
1	Libro prueba uno	2025-09-08	2025-09-29	ABIERTO
2	Libro prueba dos	2025-09-02	2025-09-13	CERRADO

4. **Top de autores por cantidad de libros registrados**
Columnas: autor, cantidad_libros. Ordenar descendente.

```
SELECT
    a.nombre AS autor,
    COUNT(l.id_libro) AS cantidad_libros
FROM Autor a
LEFT JOIN Libro l ON a.id_autor = l.id_autor
GROUP BY a.id_autor, a.nombre
ORDER BY cantidad_libros DESC;
```

Output Result 9

autor	cantidad_libros
Pepita	1
Pepito	1

5. **Disponibilidad actual de cada libro**
Columnas: titulo, stock. Filtrar libros con stock = 0 (sin unidades disponibles).

TALLER BASES DE DATOS
ING. WILLIAM ALEXANDER MATALLANA PORRAS

```
acion 72
eger 73 ✓ SELECT titulo, stock
er 74 FROM Libro
75 WHERE stock = 0;
76
77
```

Output prueba.public.libro x

titulo stock

sql 40

6. **Libros prestados actualmente (no devueltos)**

Columnas: titulo, usuario, fecha_prestamo, fecha_devolucion_max.

(Pista: `prestamo.estado='ABIERTO'` o `fecha_devuelto IS NULL` según tu regla.)

```
78 ✓ SELECT
79     l.titulo,
80     u.nombre AS usuario,
81     p.fecha_prestamo,
82     p.fecha_devolucion_max
83 FROM Prestamo p
84 INNER JOIN Libro l ON p.id_libro = l.id_libro
85 INNER JOIN Usuario u ON p.id_usuario = u.id_usuario
86 WHERE p.estado = 'ABIERTO'
87     OR p.fecha_devuelto IS NULL;
88
89
```

Output Result 11 ×

	titulo	usuario	fecha_prestamo	fecha_devolucion_max
1	Libro prueba uno	Nombre 2	2025-09-22	2025-09-30
2	Libro prueba uno	Nombre 1	2025-09-08	2025-10-15

7. **Usuarios con cantidad de préstamos abiertos**

Columnas: usuario, prestamos_abiertos. (Agrupar por usuario.)

```
90 ✓ SELECT
91     u.nombre AS usuario,
92     COUNT(p.id_prestamo) AS prestamos_abiertos
93 FROM Usuario u
94 INNER JOIN Prestamo p ON u.id_usuario = p.id_usuario
95 WHERE p.estado = 'ABIERTO'
96     OR p.fecha_devuelto IS NULL
97 GROUP BY u.id_usuario, u.nombre
98 ORDER BY prestamos_abiertos DESC;
99
100
```

Output Result 12 ×

	usuario	prestamos_abiertos
1	Nombre 1	1
2	Nombre 2	1

8. **Búsqueda por texto (título o autor)**

Dado un término, mostrar titulo, autor, anio_publicacion donde el término aparezca

en título o nombre de autor (insensible a mayúsculas).

```
✓ SELECT
    l.titulo,
    a.nombre AS autor,
    l.anio_publicacion
FROM Libro l
INNER JOIN Autor a ON l.id_autor = a.id_autor
WHERE l.titulo ILIKE '%prueba%'
    OR a.nombre ILIKE '%pep%';
```

Output Result 13 x

titulo ▼	autor ▼	anio_publicacion ▼
Libro prueba uno	Pepito	2001
Libro prueba dos	Pepita	2005

9. Préstamos vencidos

Listar usuario, título, días_atraso cuando `CURRENT_DATE > fecha_devolucion_max` y el préstamo siga abierto.

```
109 ✓ SELECT
110     u.nombre AS usuario,
111     l.titulo,
112     CURRENT_DATE - p.fecha_devolucion_max AS dias_atraso
113 FROM Prestamo p
114 INNER JOIN Usuario u ON p.id_usuario = u.id_usuario
115 INNER JOIN Libro l ON p.id_libro = l.id_libro
116 WHERE (p.estado = 'ABIERTO' OR p.fecha_devuelto IS NULL)
117 AND CURRENT_DATE > p.fecha_devolucion_max;
```

Output Result 14 ×

usuario ▾ ÷ titulo ▾ ÷ dias_atraso ▾ ÷

10. Libros por década de publicación

Mostrar década (ej. 1990s, 2000s) y cantidad de libros. (Tip: agrupa por $\text{anio_publicacion}/10$.)

```
118
119 ✓ SELECT
120     (FLOOR(l.anio_publicacion / 10) * 10)::TEXT || 's' AS decada,
121     COUNT(*) AS cantidad_libros
122 FROM Libro l
123 GROUP BY FLOOR(l.anio_publicacion / 10)
124 ORDER BY decada;
```

Output Result 16 ×

decada ▾ ÷	cantidad_libros ▾ ÷
1 2000s	2